

SubMagic Avatar POC

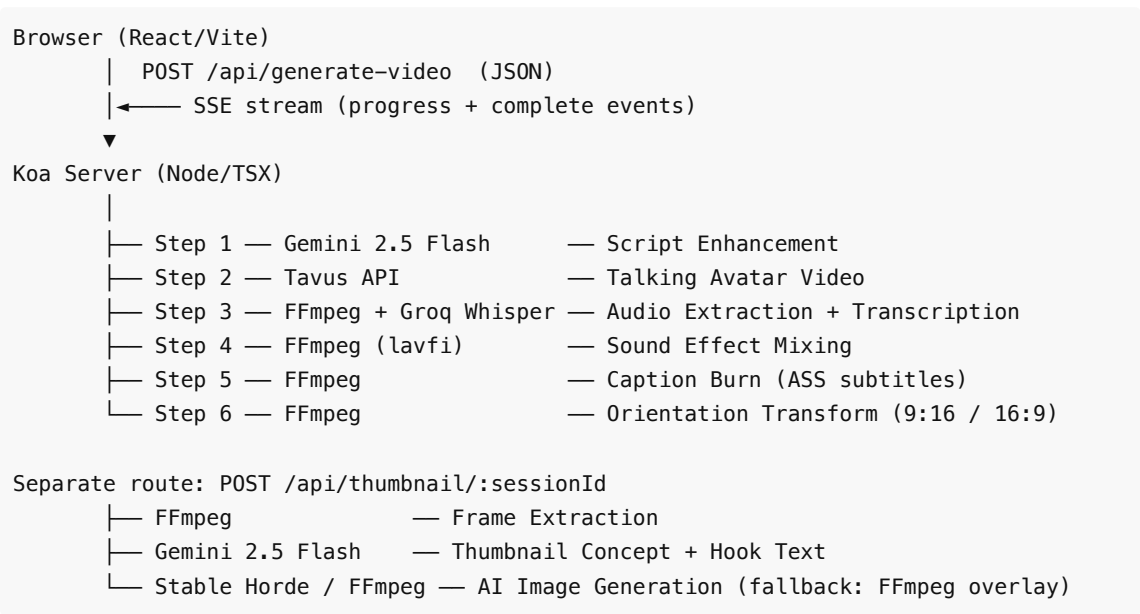
Overview

A proof-of-concept pipeline that turns a text script into a fully produced short-form video featuring a talking avatar — with AI-enhanced narration, lip-synced video, word-level captions, optional sound effects, and an AI-generated thumbnail.

Stack: Vite + React 19 (frontend) · Koa (backend) · TypeScript end-to-end

Delivery: Server-Sent Events (SSE) for real-time progress streaming

Architecture



Pipeline Steps

Step 1 — Script Enhancement (Gemini 2.5 Flash)

The user's raw script is sent to Gemini with:

- **Emotion** (neutral / happy / excited / motivational / professional / sad)
- **Duration target** (15s / 30s / 60s / 90s → maps to word count: 37 / 75 / 150 / 225 words)

Gemini rewrites the script to match the target length and inject the correct emotional tone. If Gemini is unavailable (503), the original script is used as a fallback so the pipeline never blocks.

Token usage is captured from `response.usageMetadata` and returned alongside the text.

Step 2 — Talking Avatar (Tavus)

The enhanced script is submitted to the Tavus `/v2/videos` API with a pre-configured replica ID (mapped per avatar: avatar1–4). The server polls every 6 seconds until the video status is `"ready"` (timeout: 10 minutes), then downloads the MP4.

Required env vars: `TAVUS_API_KEY` , `TAVUS_REPLICA_ID_AVATAR1 – AVATAR4` (or `TAVUS_REPLICA_ID` as fallback).

Step 3 — Transcription (Groq Whisper)

Audio is extracted from the avatar MP4 via FFmpeg (`pcm_s16le` WAV). Groq's `whisper-large-v3` model transcribes it with `verbose_json` format, returning:

- Word-level timestamps (used for caption positioning and sound effect alignment)
- Segment-level timestamps

Step 4 — Sound Effects (FFmpeg lavfi)

Three effect types are supported, synthesised entirely via FFmpeg's `aevalsrc` / `anoisesrc` filters (no audio files needed):

Effect	Generator	Character
pop	Damped sine 1 kHz, 120ms	Snappy punch
ding	Damped sine 1.5 kHz, 300ms	Bright bell
whoosh	Bandpass white noise, 250ms	Swoosh

User-selected words (by script position) are matched to transcription timestamps and mixed into the video at 40% volume via `amix` .

Step 5 — Caption Burn (FFmpeg + ASS)

Word-level timestamps from Groq are converted to an `.ass` subtitle file with three style presets:

Style	Character
viral	Large cyan bold captions, keyword highlights
professional	Small white subtitles, minimal
creator	Word-by-word animated captions

All style properties (color, font size, outline, bold/italic) are fully customisable via the UI. FFmpeg burns the subtitles with `libx264` re-encode.

Step 6 — Orientation Transform (FFmpeg)

- **Landscape (16:9)**: stream copy, no re-encode
- **Portrait (9:16)**: scale to 1080px width, pad to 1080x1920 with black bars

Thumbnail Generation (Separate Route)

Triggered from the UI after the video is ready. Three stages:

1. **Frame extraction** — a clean frame is pulled from `avatar.mp4` at a style-specific timestamp (33%–60% through)
2. **Gemini Vision concept** — the frame + script are sent to Gemini 2.5 Flash (multimodal) to generate a `hookLine` , `subLine` , and a detailed Stable Diffusion prompt describing the person and layout

3. Image generation:

- **Primary:** Stable Horde (free community FLUX/SDXL, anonymous key, 512×320, 20 steps, 30s timeout)
- **Fallback:** FFmpeg drawtext / drawbox overlay — applies color grading, vignette, and renders hook text directly onto the extracted frame

Five thumbnail styles: engaging , normal , cinematic , professional , dynamic .

AI Token Usage

Token usage is tracked per Gemini call and surfaced in the UI after generation completes.

What's tracked

Call	Where	Returns
enhanceScript	gemini.ts	{ text, tokens }
analyzeSoundEffects	gemini.ts	{ effects, tokens } — currently unused in main pipeline
generateConcept (thumbnail)	thumbnail.ts	Logged to console only

Token counts come from response.usageMetadata :

- promptTokenCount → input tokens
- candidatesTokenCount → output tokens
- totalTokenCount → total (includes any thinking tokens for Flash 2.5)

Typical usage per video

Step	Input tokens	Output tokens	Notes
Script Enhancement	~300–600	~75–250	Scales with script length + duration
Thumbnail Concept	~800–1500	~200–400	Includes base64 frame image

Groq Whisper and macOS say (TTS) are not token-based — they bill by audio duration.
Tavus bills per second of generated video.

Services & APIs

Service	Used for	Auth
Gemini 2.5 Flash (@google/genai)	Script enhancement, thumbnail concept	GEMINI_API_KEY
Tavus	Talking avatar video generation	TAVUS_API_KEY
Groq Whisper (groq-sdk)	Audio transcription with word timestamps	GROQ_API_KEY
macOS say + FFmpeg	Text-to-speech (AIFF → MP3)	None (local)

Stable Horde	Free AI thumbnail image generation	Anonymous key
FFmpeg (fluent-ffmpeg)	Audio extraction, caption burn, SFX mix, orientation	None (local binary)

Environment Variables

```
GEMINI_API_KEY=...
TAVUS_API_KEY=...
TAVUS_REPLICA_ID=...          # fallback for all avatars
TAVUS_REPLICA_ID_AVATAR1=...  # per-avatar override
TAVUS_REPLICA_ID_AVATAR2=...
TAVUS_REPLICA_ID_AVATAR3=...
TAVUS_REPLICA_ID_AVATAR4=...
GROQ_API_KEY=...
```

Copy `.env.example` to `.env` and fill in values.

Running Locally

```
npm install
npm run dev          # starts both Vite (port 5173) and Koa (port 3001) concurrently
```

Production build:

```
npm run build        # Vite → dist/
node -r dotenv/config server/index.ts  # serve dist/ + API
```

Known Limitations / POC Scope

- **macOS-only TTS** — uses `say` CLI. Replacing with OpenAI TTS or ElevenLabs would make it cross-platform and improve voice quality.
- **Tavus latency** — avatar rendering takes 60–180 seconds. No queue management or webhooks; pure polling.
- **Single-user sessions** — no auth, no persistence. Session files live in OS temp dir and are deleted on "Generate Another".
- **Stable Horde availability** — anonymous free tier has variable queue times and low resolution (512×320). A paid image API would be more reliable.
- **No transcript correction** — Groq Whisper occasionally mishears words; caption errors are not correctable in the UI.
- **analyzeSoundEffects** — the Gemini-based auto sound effect placement function is implemented but not wired into the main pipeline. Currently sound effects are user-selected only.